

Task 1

编写的求解斐波那契数列的 RISC-V 汇编程序核心代码如下：

```
main:
    beqz a0, end_fib      # 若 a0 = 0, 直接返回结果为 0 = a0

    li t1, 1              # F(1) = 1, 同时用于 n = 1 的条件判定
    la s0, fib_array      # s0 -> 存储基地址 (每次存储后加 4)
    sw t1, 0(s0)          # 存储 F(1)
    addi s0, s0, 4         # 指向下一个存储位置
    beq a0, t1, end_fib    # 若 a0 = 1, 直接返回结果为 1 = a1

    blt a0, zero, err_proc # 若 a0 < 0, 进入错误处理程序

    # 赋予初值
    # 在循环结束时 t0 表示 F(t2-1), t1 表示 F(t2)
    # t2 作为计数器
    li t0, 0
    li t2, 1

loop:
    beq t2, a0, end_loop   # 若 t2 == n, 则退出循环

    add t3, t0, t1         # 计算 F(i+1) = F(i-1) + F(i)
    mv t0, t1              # t0 = t1
    mv t1, t3              # t1 = t3

    addi t2, t2, 1         # 计数器自增
    sw t1, 0(s0)           # 存储当前 F(t2)
    addi s0, s0, 4         # 指向下一个存储位置
    j loop

err_proc:
    li a0, -1              # 令 a0 = -1 作为报错状态
    j end_fib

end_loop:
    mv a0, t1              # 循环结束时, t2 = n, t1 寄存器的值即为 F(n)
```

首先检查输入的参数 n （存储在 $a0$ 中）是否为 0 或 1，若是则直接结束并返回结果为 1；同时利用 `blt` 判断参数是否小于 0，若为负数则跳转至 `err_proc` 处理异常设定返回值 $a0 = -1$ 。当 $n > 1$ 时，使用寄存器 $t0$ 和 $t1$ 维护数列中相邻的两个状态项（初始时令 $t0 = 0, t1 = 1$ ）；使用 $t2$ 作为迭代计数器（初值为 1）。

在每轮循环初，判断 $t2(=i) == a0(=n)$ 是否成立，当计数器达到要求的 n 时跳出循环；在循环内利用 `add` $t3, t0, t1$ 计算出新一项的数据，将当前的 $t1 = F(t2)$ 存入内存，再迭代更新寄存器状态（ $t0 = t1, t1 = t3$ ），并将 $t2$ 自增。

在循环结束时， $t1$ 中的值即为所求的 $F(n)$ ，将其转存入返回值寄存器 $a0$ 并触发 30 号系统调用，结束求解程序。

Task 2

编写的执行冒泡排序的 RISC-V 汇编程序核心代码如下：

```
main:                                # 程序入口点
    # 获取开始时间
    li a7, 30
    ecall
    mv s0, a0                        # 将开始时间的低 32 位存入 s0

    la a0, nums                      # la(Load Address): 加载 nums 标签的地址到寄存器 a0
    lw t0, length                    # 加载数组长度
    addi t0, t0, -1                  # 最后一个元素的索引，同时也是外层循环的上限

    li t1, 0                         # 外层循环变量，从 0 递增到 127
    # li t2, 127                     # 内层循环变量，从 127 递减到 t1+1
outer_loop:
    bge t1, t0, sort_finish          # 循环跳出条件: t1 >= 127

    li t2, 127                       # 内层循环变量，从 127 递减到 t1+1
inner_loop:
    bge t1, t2, inner_finish          # 循环跳出条件: t2 <= t1

    add t4, a0, t2
    lb t5, 0(t4)                     # 读取 nums[t2]
    lb t6, -1(t4)                    # 读取 nums[t2-1]
    blt t5, t6, no_swap               # 若 nums[t2] < nums[t2-1], 则进行交换
    # 如需改为降序, 在此处交换 t5 和 t6 的参数位置即可
    # 如需改为无符号数比较, 在此处将 blt 改为 bltu 即可

    sb t5, -1(t4)
    sb t6, 0(t4)

no_swap:
    addi t2, t2, -1                  # 内层循环变量自减
    j inner_loop

inner_finish:
    addi t1, t1, 1                   # 外层循环变量自增
    j outer_loop
```

按照冒泡排序的算法思路，程序分为外层循环和内层循环，并使用了两个排序辅助变量 `t1`，`t2`。第 `t1+1` 次外层循环结束后，下标 `0` 到 `t1` 范围内的数据已排序完成。

执行时间的测试方法

- 在排序开始前，使用系统调用号为 `30` 的 `ecall` 获取系统时间。该系统调用返回当前时间的毫秒级时间戳，并将低 32 位保存在 `a0` 寄存器中。
- 为保证时间测试的外套代码不影响程序主体的功能，在执行系统调用后立刻将时间数据转存到寄存器 `s0` 中，保证后续可以通过 `la a0, nums` 正常加载数组基地址

- 在排序结束后，再次调用 30 号系统调用获取系统时间，并通过通过 `sub s1, a0, s0` 计算得到程序执行用时。